

PRACTICAL -1

1. An electricity board charges the following rates to domestic users to discourage large consumption of energy.

For the first 100 units: - 60 P per unit

For the next 200 units: -80 P per unit

Beyond 20 units: -90 P per unit

All users are charged a minimum of Rs 50 if the total amount is more than Rs 20 then an additional surcharge of 15% is added. Implement a C++ program to read the names of users and number of units consumed and display the charges with names.

Name-ABHINAV SINGH

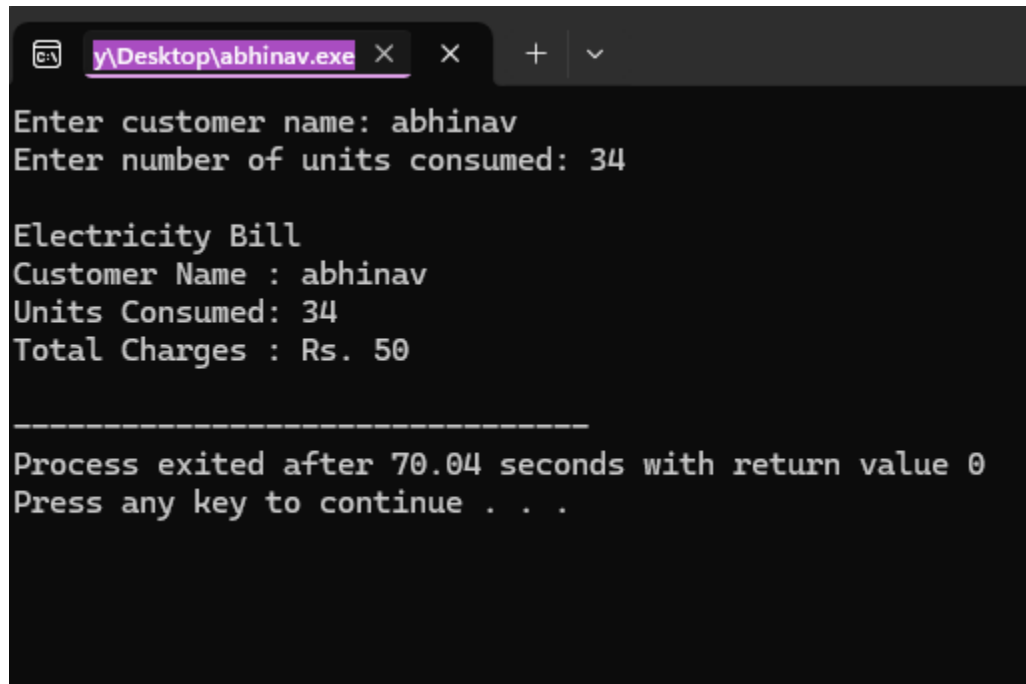
Section-E2

Roll no.-2

Source Code-

```
#include <iostream>
#include <string>
using namespace std;
int main(){
    string n;int u;double b=0;
    cout<<"Enter customer name: ";getline(cin,n);
    cout<<"Enter number of units consumed: ";cin>>u;
    if(u<=100){b=u*0.60;}
    else if(u<=200){b=(100*0.60)+((u-100)*0.80);}
    else{b=(100*0.60)+(200*0.80)+((u-200)*0.90);}
    if(b<50){b=50;}
    if(b>20){b=b+(b*0.15);}
    cout<<"\nElectricity Bill\n";
    cout<<"Customer Name : "<<n<<endl;
    cout<<"Units Consumed: "<<u<<endl;
    cout<<"Total Charges : Rs. "<<b<<endl;
    return 0;}
```

OUTPUT:



```
y:\Desktop\abhinav.exe x x + v
Enter customer name: abhinav
Enter number of units consumed: 34

Electricity Bill
Customer Name : abhinav
Units Consumed: 34
Total Charges : Rs. 50

-----
Process exited after 70.04 seconds with return value 0
Press any key to continue . . .
```

2. Construct a C++ program that removes a specific character from a given string and return the updated string.

Typical Input: computer science is the future

Typical Output: compuer science is he fuure.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

Source code-

```
#include <iostream>

#include <string>

using namespace std;

string removeChar(string str, char ch) {
    string result = "";
    for (int i = 0; i < str.length(); i++) {
        if (str[i] != ch) {
            result += str[i]; // keep the character if it's not the one to remove
        }
    }
    return result;
}

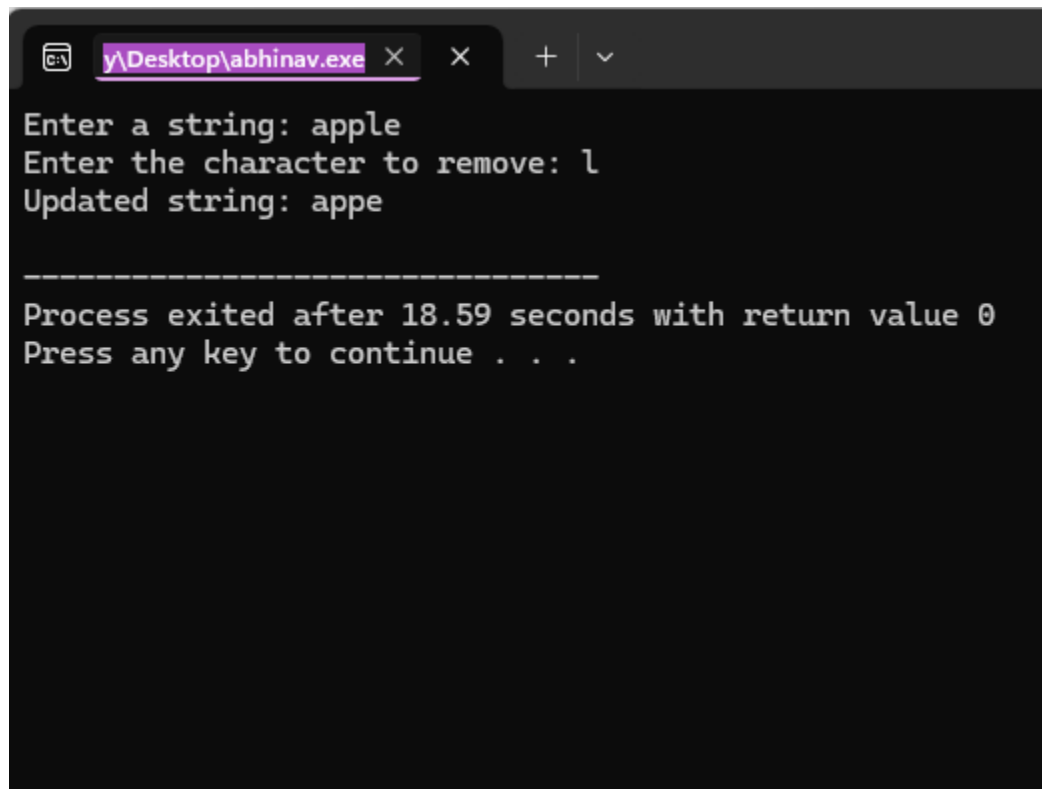
int main() {
    string input;
    char ch;

    cout << "Enter a string: ";
    getline(cin, input);

    cout << "Enter the character to remove: ";
    cin >> ch;

    string updated = removeChar(input, ch);
    cout << "Updated string: " << updated << endl;
    return 0; }
```

OUTPUT:



```
y\Desktop\abhinav.exe x x + v
Enter a string: apple
Enter the character to remove: l
Updated string: appe

-----
Process exited after 18.59 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-2

3.Implement a C++ program to find the non-repeating characters in string.

Typical Input: graphic era university.

Typical Output: c g h n p s t u v y

Name-ABHINAV SINGH

Section-E2

Roll no.-2

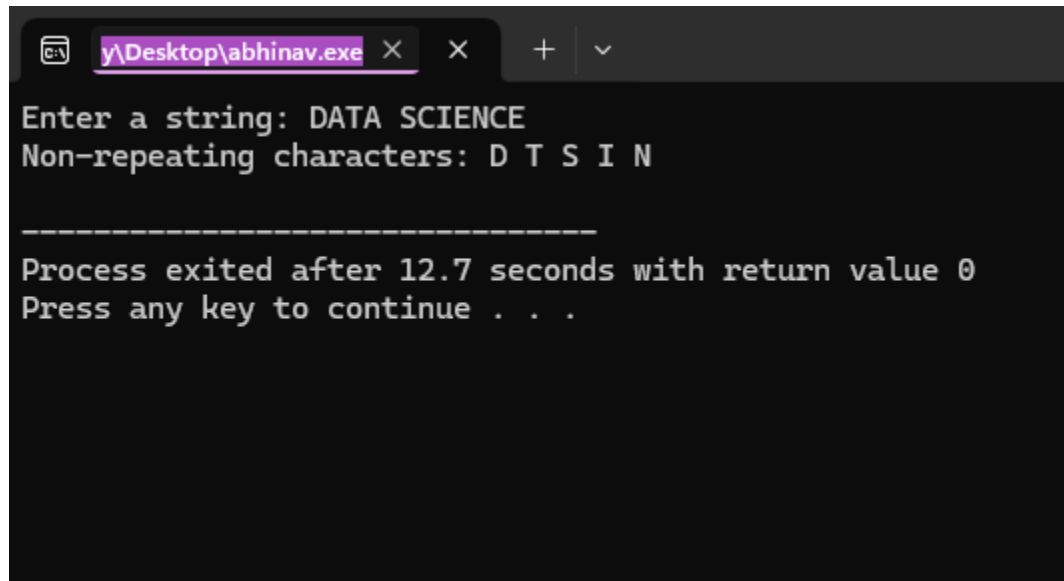
Source Code:

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string str;
    cout << "Enter a string: ";
    getline(cin, str);
    cout << "Non-repeating characters: ";

    for (int i = 0; i < str.size(); i++) {
        char ch = str[i];
        if (ch == ' ') continue; // skip spaces

        int count = 0;
        for (int j = 0; j < str.size(); j++) {
            if (str[j] == ch) count++;
        } if (count == 1) {
            cout << ch << " "; } }
    cout << endl;
    return 0;}
```

OUTPUT:



```
y:\Desktop\abhinav.exe x x + v
Enter a string: DATA SCIENCE
Non-repeating characters: D T S I N

-----
Process exited after 12.7 seconds with return value 0
Press any key to continue . . .
```

4. You are given an array A of N elements. Now you need to choose the best index of this array A. An index of the array is called best if the special sum of this index is maximum across the special sum of all the other indices.

To calculate the special sum for any index i, you pick the first element that is A[i] and add it to your sum. Now you pick next two elements i.e. A[i + 1] and A[i + 2] and add both of them to your sum. Now you will pick the next 3 elements and this continues till the index for which it is possible to pick the elements. For example:

If our array contains 10 elements and you choose index to be 3 then your special sum is denoted by -

$(A[3]) + (A[4] + A[5]) + (A[6] + A[7] + A[8]),$

beyond this its not possible to add further elements as the index value will cross the value 10.

Find the best index and in the output print its corresponding special sum. Note that there may be more than one best indices but you need to only print the maximum special sum.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

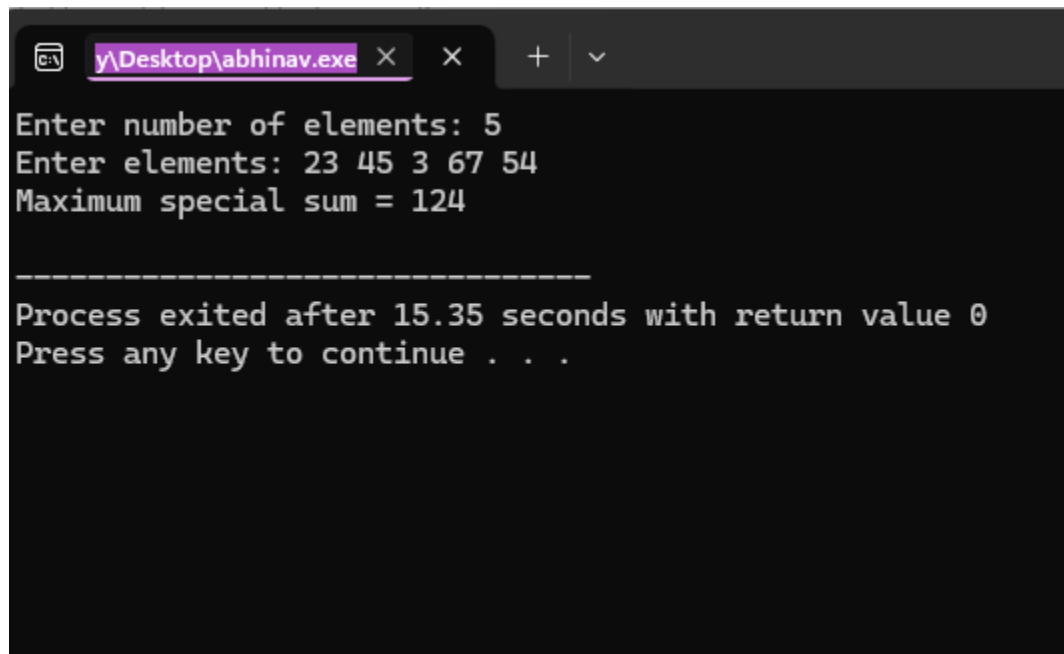
SOURCE CODE:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    int a[1000]; // assuming max size 1000, you can increase if needed
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) cin >> a[i];
    int best = 0;
    for (int i = 0; i < n; i++) {
        int sum = 0, k = 1, j = i;
        while (j + k <= n) {
            for (int x = 0; x < k; x++) {
                sum += a[j + x];
            }
            j += k;
            k++;
        }
        if (sum > best) best = sum;
    } cout << "Maximum special sum = " << best << endl;
    return 0;}
```

OUTPUT:



```
y:\Desktop\abhinav.exe x x + v
Enter number of elements: 5
Enter elements: 23 45 3 67 54
Maximum special sum = 124

-----
Process exited after 15.35 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-5

5.Implement a C++ program to demonstrate the concept of data abstraction using the concept of Class and Objects.

Name-ABHINAV SINGH

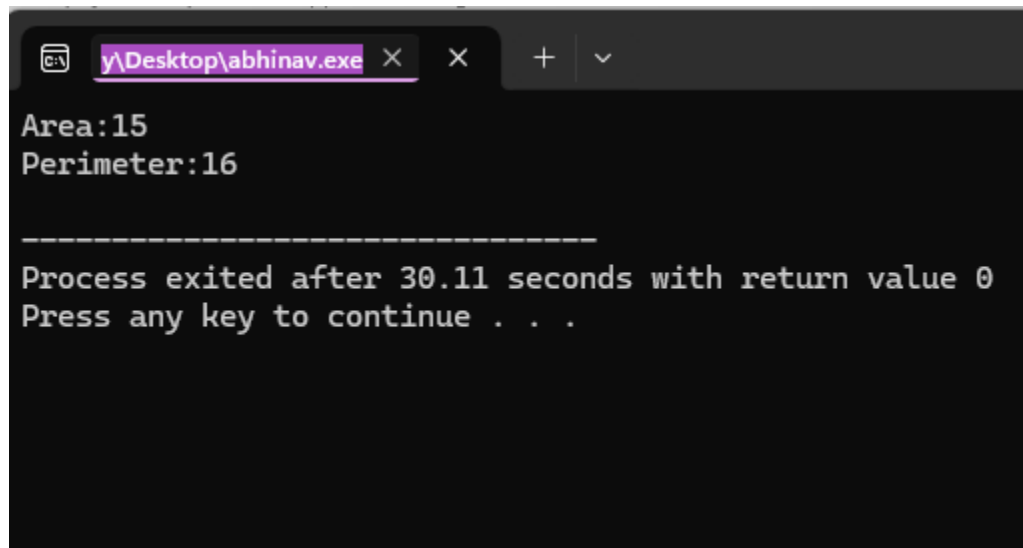
Section-E2

Roll no.-2

SOURCE CODE-

```
#include<iostream>
using namespace std;
class Rectangle{
private:int l,b;
public:
void setData(int x,int y){l=x;b=y;}
int area(){return l*b;}
int perimeter(){return 2*(l+b);}
};
int main(){
Rectangle r;
r.setData(5,3);
cout<<"Area:"<<r.area()<<endl;
cout<<"Perimeter:"<<r.perimeter()<<endl; }}
```

OUTPUT:



```
Area:15
Perimeter:16

-----
Process exited after 30.11 seconds with return value 0
Press any key to continue . . .
```

6. Define a class Hotel in C++ with the following specifications

Private members

- Rno Data member to store room number
- Name Data member to store customer name
- Tariff Data member to store per day charges
- NOD Data member to store number of days of stay
- CALC() Function to calculate and return amount as $\text{NOD} * \text{Tariff}$, and if the value of $\text{days} * \text{Tariff} > 10000$, then total amount is $1.05 * \text{days} * \text{Tariff}$.

Public members

- Checkin() Function to enter the content Rno, Name, Tariff and NOD
- Checkout() Function to display Rno, Name, Tariff, NOD and Amount (amount to be displayed by calling function) CALC().

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Hotel {
private:
    int roomNo;
    string name;
    int tariff;
    int days;

    int calc() {
        int amt = days * tariff;
        if (amt > 10000) amt = amt * 1.05;
        return amt;
    }

public:
    void checkin() {
        cout << "Enter Room No: ";
```

```

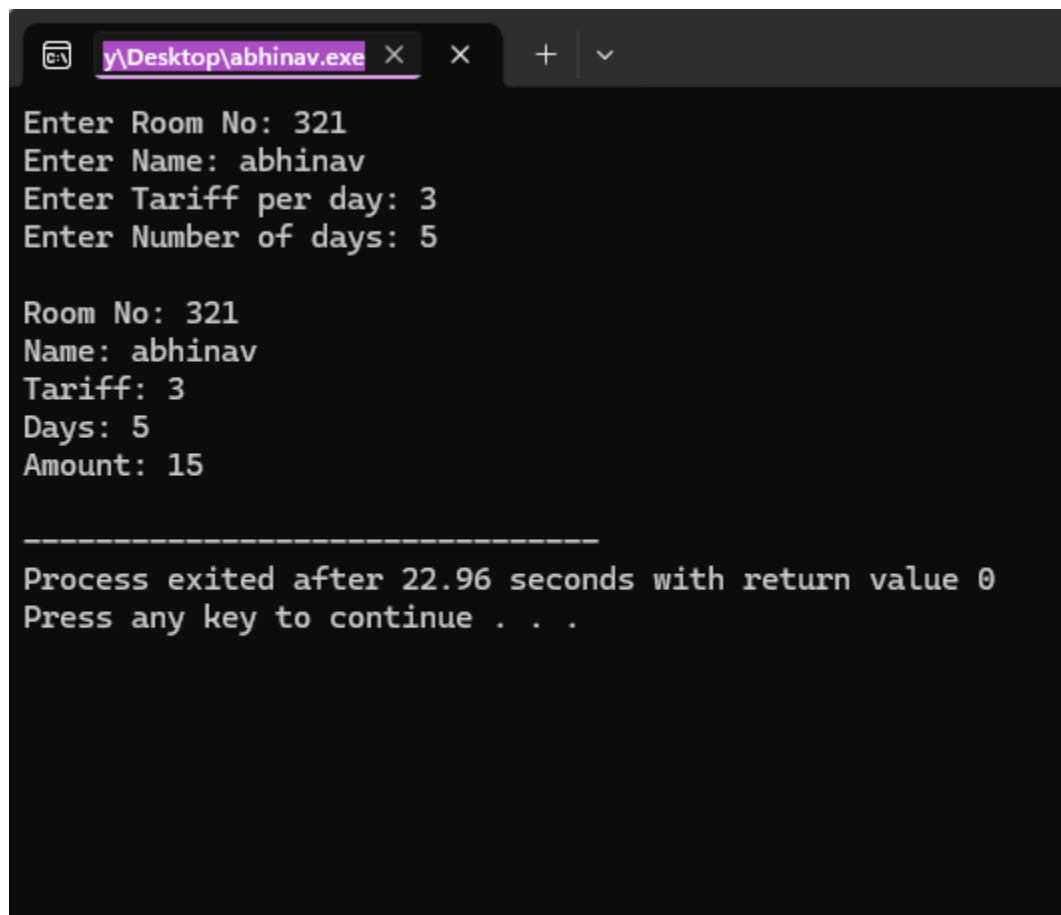
        cin >> roomNo;
        cin.ignore();
        cout << "Enter Name: ";
        getline(cin, name);
        cout << "Enter Tariff per day: ";
        cin >> tariff;
        cout << "Enter Number of days: ";
        cin >> days;
    }

    void checkout() {
        cout << "\nRoom No: " << roomNo;
        cout << "\nName: " << name;
        cout << "\nTariff: " << tariff;
        cout << "\nDays: " << days;
        cout << "\nAmount: " << calc() << endl;
    }
};

int main() {
    Hotel h;
    h.checkin();
    h.checkout();
    return 0;}

```

OUTPUT:



```
y:\Desktop\abhinav.exe x x + v
Enter Room No: 321
Enter Name: abhinav
Enter Tariff per day: 3
Enter Number of days: 5

Room No: 321
Name: abhinav
Tariff: 3
Days: 5
Amount: 15

-----
Process exited after 22.96 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-4

7.Implement a Program in C++ by defining a class to represent a bank account.

Include the following:

Data Members

- Name of the depositor
- Account number
- Type of account (Saving, Current etc.)
- Balance amount in the account

Member Functions

- To assign initial values
- To deposit an amount
- To withdraw an amount after checking the balance
- To display name and balance

Name-ABHINAV SINGH

Section-E2

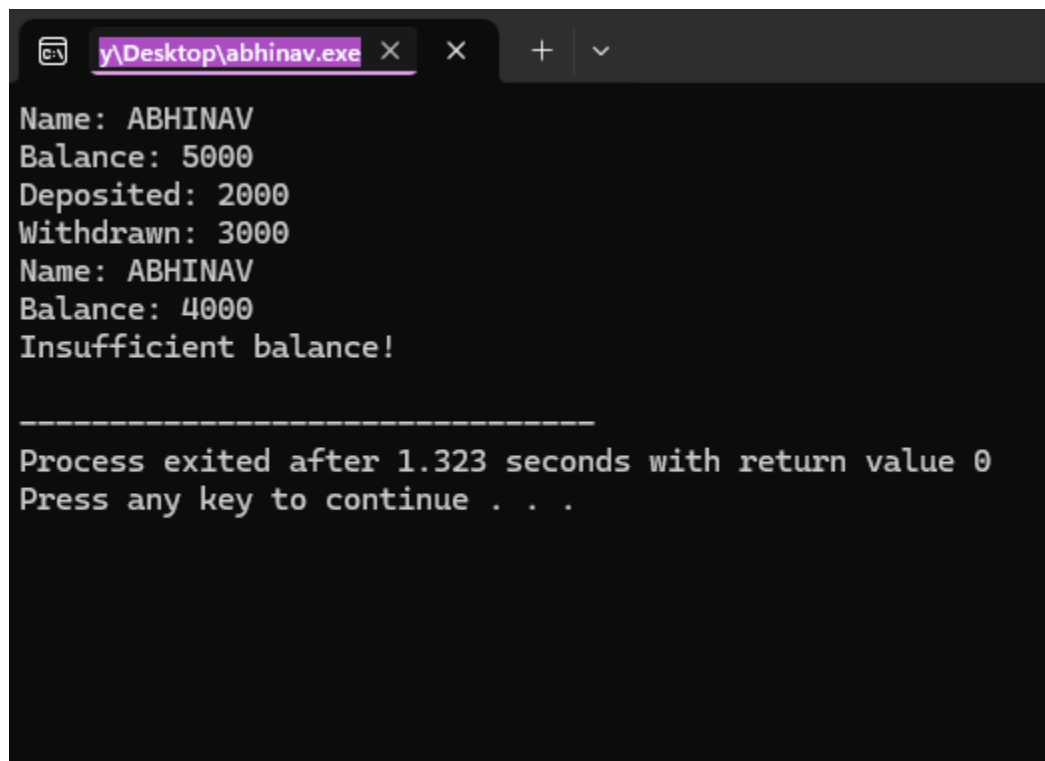
Roll no.-2

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;
class Bank {
private:
    string name;
    int accNo;
    string type;
    int balance;
public:
    void init(string n,int a,string t,int b){
        name=n;
        accNo=a;
        type=t;
        balance=b;
    }
    void deposit(int amt){
        balance+=amt;
        cout<<"Deposited: "<<amt<<endl;
```

```
}  
void withdraw(int amt){  
    if(amt<=balance){  
        balance-=amt;  
        cout<<"Withdrawn: "<<amt<<endl;  
    } else {  
        cout<<"Insufficient balance!"<<endl;  
    }  
}  
void display(){  
    cout<<"Name: "<<name<<endl;  
    cout<<"Balance: "<<balance<<endl;  
}  
};  
int main(){  
    Bank b;  
    b.init("Kritika",12345,"Saving",5000);  
    b.display();  
    b.deposit(2000);  
    b.withdraw(200);  
    b.display();  
    b.withdraw(6000);  
    return 0;}
```

OUTPUT:



```
y:\Desktop\abhinav.exe x x + v
Name: ABHINAV
Balance: 5000
Deposited: 2000
Withdrawn: 3000
Name: ABHINAV
Balance: 4000
Insufficient balance!

-----
Process exited after 1.323 seconds with return value 0
Press any key to continue . . .
```

8. Anna is a contender for valedictorian of her high school. She wants to know how many students (if any) have scored higher than her in the exams given during this semester.

Create a class named Student with the following specifications:

- An instance variable named scores holds a student's 5 exam scores.
- A void input () function reads 5 integers and saves them to scores.
- An int calculateTotalScore() function that returns the sum of the student's scores.

Input Format

In the void Student::input() function, you must read 5 scores from standard input and save them to your scores instance variable.

Output Format

In the int Student::calculateTotalScore() function, you must return the student's total grade (the sum of the values in scores).

The code in the editor will determine how many scores are larger than Anna's and print that number to the console.

Sample Input

The first line contains n, the number of students in Anna's class. The n subsequent lines contain each student's 5 exam grades for this semester.

```
3
2 40 45 10 10
40 40 40 10 10
50 20 2 10 10
```

Sample Output

```
1
Name-ABHINAV SINGH
Section-E2
Roll no.-2
```

SOURCE CODE:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
class Student {
```

```
public:
```

```
    int scores[5]; // holds 5 exam scores
```

```
    void input() {
```

```
        cout << "Enter 5 exam scores: ";
```

```
        for (int i = 0; i < 5; i++) {
```

```
            cin >> scores[i];
```

```
        }
```

```
    }
```

```
    int calculateTotalScore() {
```

```
        int sum = 0;
```

```
        for (int i = 0; i < 5; i++) {
```

```
            sum += scores[i];
```

```
        }
```

```
        return sum;
```

```
    }
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    cout << "Enter number of students: ";
```

```
cin >> n;
```

```
vector<Student> students(n);
```

```
for (int i = 0; i < n; i++) {  
    cout << "\n--- Student " << i + 1 << " ---" << endl;  
    students[i].input();  
}
```

```
int annaScore = students[0].calculateTotalScore(); // first student is Anna  
int count = 0;
```

```
for (int i = 1; i < n; i++) {  
    if (students[i].calculateTotalScore() > annaScore) {  
        count++;  
    }  
}
```

```
cout << "\nNumber of students who scored higher than Anna: " << count << endl;
```

```
return 0;  
}
```

OUTPUT:

```
y:\Desktop\abhinav.exe x x + v
Enter number of students: 5

--- Student 1 ---
Enter 5 exam scores: 23 76 98 78 65

--- Student 2 ---
Enter 5 exam scores: 43 78 65 12 90

--- Student 3 ---
Enter 5 exam scores: 90 99 98 97 96

--- Student 4 ---
Enter 5 exam scores: 65 64 67 78 98

--- Student 5 ---
Enter 5 exam scores: 12 13 14 15 16

Number of students who scored higher than Anna: 2

-----
Process exited after 69.94 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-5

9. Construct a Program in C++ to show the working of function overloading (compile time polymorphism) by using a function named calculateArea() to calculate area of square, circle, rectangle and triangle using different signatures as required.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

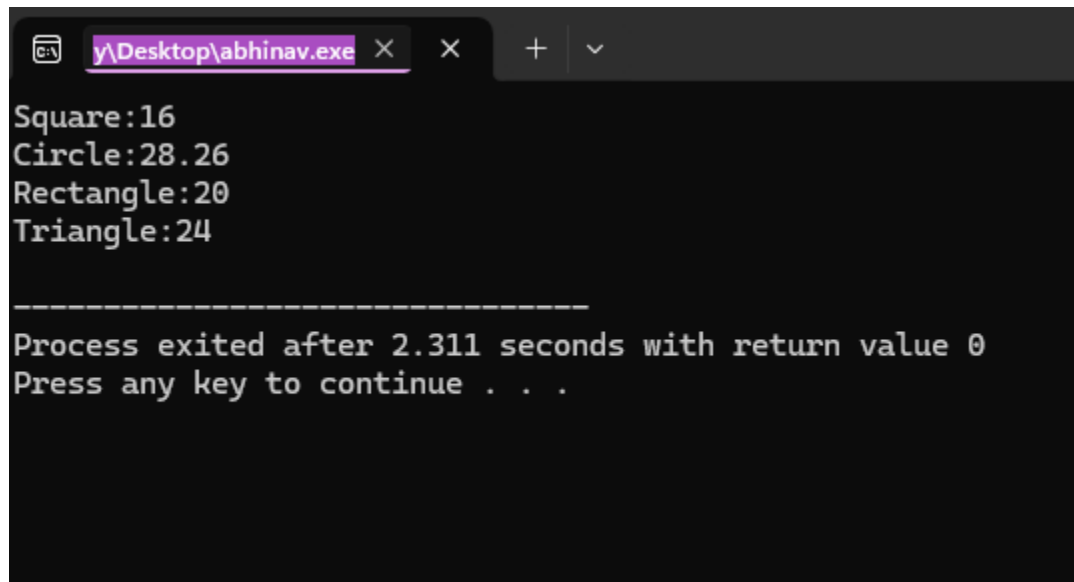
```
#include<iostream>

using namespace std;

class Area{
public:
    int calculateArea(int a){return a*a;}           //square
    float calculateArea(float r){return 3.14f*r*r;} //circle
    int calculateArea(int l,int b){return l*b;}      //rectangle
    float calculateArea(float b,float h){return 0.5f*b*h;} //triangle
};

int main(){
    Area x;
    cout<<"Square:"<<x.calculateArea(4)<<endl;
    cout<<"Circle:"<<x.calculateArea(3.0f)<<endl;
    cout<<"Rectangle:"<<x.calculateArea(4,5)<<endl;
    cout<<"Triangle:"<<x.calculateArea(6.0f,8.0f)<<endl;
}
```

OUTPUT:



A screenshot of a Windows command prompt window. The title bar shows the file path 'y:\Desktop\abhinav.exe' and standard window controls. The command prompt displays the following output:

```
Square:16  
Circle:28.26  
Rectangle:20  
Triangle:24  
  
-----  
Process exited after 2.311 seconds with return value 0  
Press any key to continue . . .
```

10. Create a class called Invoice that a hardware store might use to represent an invoice for an item sold at the store. An Invoice should include four pieces of information as instance. **Data Members** –

- partNumber (type String)
- partDescription (type String)
- quantity of the item being purchased (type int)
- price_per_item (type double)

Your class should have a constructor that initializes the four instance variables. Provide a set and a get method for each instance variable. In addition, provide a method named `getInvoiceAmount()` that calculates the invoice amount (i.e., multiplies the quantity by the price per item), then returns the amount as a double value. If the quantity is not positive, it should be set to 0. If the price per item is not positive, it should be set to 0.0. Write a test application named `invoiceTest` that demonstrates class Invoice's capabilities.

Name-ABHINAV SINGH
Section-E2
Roll no.-2

SOURCE CODE:

```
#include<iostream>

using namespace std;

class Invoice {
    string partNumber;
    string partDescription;
    int quantity;
    float price_per_item;
public:
    Invoice(string pn, string pd, int q, float p) {
        partNumber = pn;
        partDescription = pd;
        quantity = q > 0 ? q : 0;
        price_per_item = p > 0 ? p : 0.0;
    }
```

```

void setPartNumber(string pn) { partNumber = pn; }
string getPartNumber() { return partNumber; }
void setPartDescription(string pd) { partDescription = pd; }
string getPartDescription() { return partDescription; }
void setQuantity(int q) { quantity = q > 0 ? q : 0; }
int getQuantity() { return quantity; }
void setPricePerItem(float p) { price_per_item = p > 0 ? p : 0.0; }
float getPricePerItem() { return price_per_item; }
float getInvoiceAmount() { return quantity * price_per_item; }
};

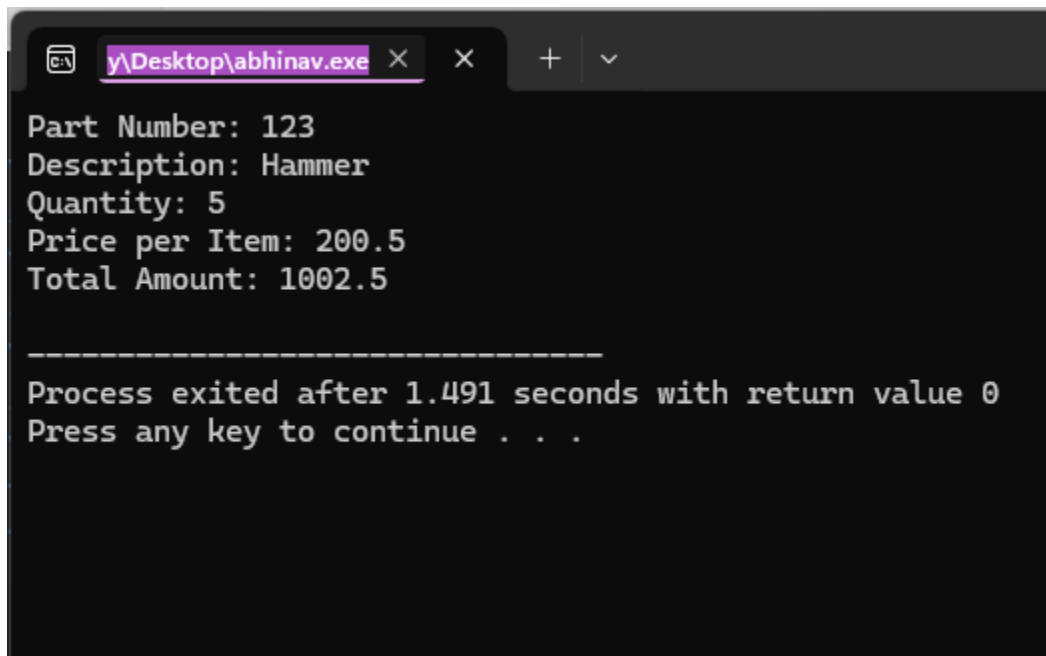
```

```

int main() {
    Invoice item("123", "Hammer", 5, 200.5);
    cout << "Part Number: " << item.getPartNumber() << endl;
    cout << "Description: " << item.getPartDescription() << endl;
    cout << "Quantity: " << item.getQuantity() << endl;
    cout << "Price per Item: " << item.getPricePerItem() << endl;
    cout << "Total Amount: " << item.getInvoiceAmount() << endl;
    return 0;
}

```

OUTPUT:



A screenshot of a Windows command prompt window. The title bar shows the file path 'y\Desktop\abhinav.exe' with standard window controls. The output text is as follows:

```
Part Number: 123
Description: Hammer
Quantity: 5
Price per Item: 200.5
Total Amount: 1002.5

-----
Process exited after 1.491 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-6

11. Imagine a tollbooth with a class called TollBooth. The two data items are of type unsigned int and double to hold the total number of cars and total amount of money collected. A constructor initializes both of these data members to 0. A member function called payingCar() increments the car total and adds 0.5 to the cash total. Another function called nonPayCar() increments the car total but adds nothing to the cash total. Finally a member function called display() shows the two totals. Include a program to test this class. This program should allow the user to push one key to count a paying car and another to count a non paying car. Pushing the ESC key should cause the program to print out the total number of cars and total cash and then exit.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include <iostream>
using namespace std;

class TollBooth {
    unsigned int totalCars;
    double totalCash;
public:
    TollBooth() {
        totalCars = 0;
        totalCash = 0.0;
    }

    void payingCar() {
        totalCars++;
        totalCash += 0.5;
    }

    void nonPayCar() {
        totalCars++;
    }

    void display() {
        cout << "\nTotal Cars: " << totalCars << endl;
        cout << "Total Cash: " << totalCash << endl;
    }
};

int main() {
    TollBooth booth;
    char choice;
```

```
cout << "Enter 'p' for Paying Car, 'n' for Non-Paying Car, 'e' to Exit:\n";

while (true) {
    cin >> choice;
    if (choice == 'p' || choice == 'P') {
        booth.payingCar();
        cout << "Paying car added!\n";
    }
    else if (choice == 'n' || choice == 'N') {
        booth.nonPayCar();
        cout << "Non-paying car added!\n";
    }
    else if (choice == 'e' || choice == 'E') {
        break;
    }
    else {
        cout << "Invalid input! Use p, n, or e.\n";
    }
}

booth.display();
return 0;
}
```

OUTPUT:

```
y\Desktop\abhinav.exe x x + v
Enter 'p' for Paying Car, 'n' for Non-Paying Car, 'e' to Exit:
p
Paying car added!
p
Paying car added!
p
Paying car added!
p
Paying car added!
n
Non-paying car added!
n
Non-paying car added!
e

Total Cars: 6
Total Cash: 2

-----
Process exited after 18.79 seconds with return value 0
Press any key to continue . . .
```

12.Create a class called Time that has separate int member data for hours, minutes and seconds. One constructor should initialize this data to 0, and another should initialize it to fixed values. A member function should display it in 11:59:59 format. A member function named add() should add two objects of type time passed as arguments. A main () program should create two initialized values together, leaving the result in the third time variable. Finally it should display the value of this third variable

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include<iostream>

using namespace std;

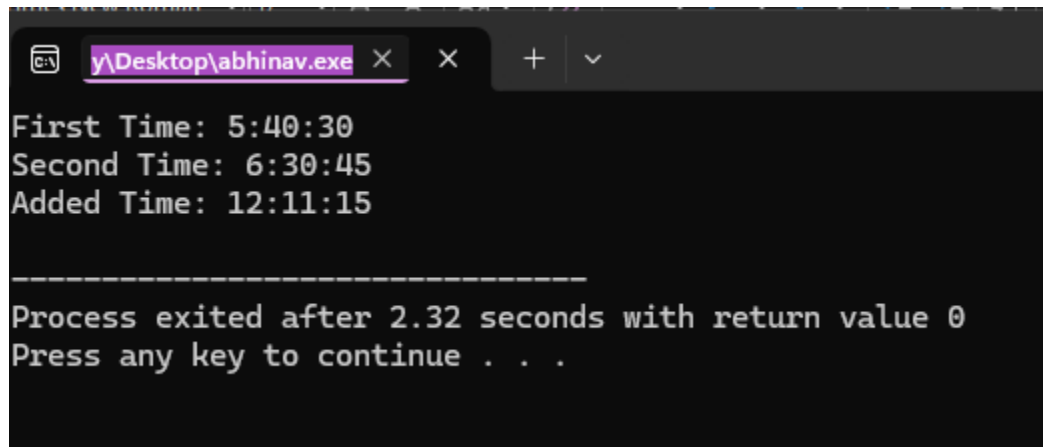
class Time { int hours, minutes, seconds;

public: Time() { hours=0; minutes=0;

        seconds=0; }
    Time(int h,int m,int s) {
        hours=h;
        minutes=m;
        seconds=s;
    } void display() {
        cout<<hours<<":"<<minutes<<":"<<seconds<<endl;
    } void add(Time t1,Time t2) {
        seconds=t1.seconds+t2.seconds;
        minutes=t1.minutes+t2.minutes+seconds/60;
        seconds=seconds%60;
        hours=t1.hours+t2.hours+minutes/60;
        minutes=minutes%60; } };

int main() {
    Time t1(5,40,2);
    Time t2(6,2,45);
    Time t3;
    t3.add(t1,t2);
    cout<<"First Time: ";
    t1.display();
    cout<<"Second Time: ";
    t2.display();
    cout<<"Added Time: ";
    t3.display();
    return 0;}
```

OUTPUT:



```
y:\Desktop\abhinav.exe x x + v
First Time: 5:40:30
Second Time: 6:30:45
Added Time: 12:11:15

-----
Process exited after 2.32 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-7

13. Create class SavingsAccount. Use a static variable annualInterestRate to store the annual interest rate for all account holders. Each object of the class contains a private instance variable savingsBalance indicating the amount the saver currently has on deposit. Provide method calculateMonthlyInterest() to calculate the monthly interest by multiplying the savingsBalance by annualInterestRate divided by 12. This interest should be added to savingsBalance. Provide a static method modifyInterestRate() that sets the annualInterestRate to a new value. Write a program to test class SavingsAccount. Instantiate two SavingsAccount objects, saver1 and saver2, with balances of Rs2000.00 and Rs200.00, respectively. Set annualInterestRate to 4%, then calculate the monthly interest and print the new balances for both savers. Then set the annualInterestRate to 5%, calculate the next month's interest and print the new balances for both savers.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include <iostream>
using namespace std;

class SavingsAccount {
private:
    double savingsBalance;
    static double annualInterestRate;

public:
    // Constructor
    SavingsAccount(double balance) {
        savingsBalance = balance;
    }

    // Calculate monthly interest and update balance
    void calculateMonthlyInterest() {
        double monthlyInterest = (savingsBalance * annualInterestRate) / 12;
        savingsBalance += monthlyInterest;
    }

    // Display current balance
    void displayBalance() {
        cout << "Balance: Rs " << savingsBalance << endl;
    }

    // Static function to modify interest rate
    static void modifyInterestRate(double newRate) {
```

```

        annualInterestRate = newRate;
    }
};

// Define static member
double SavingsAccount::annualInterestRate = 0.0;

int main() {
    SavingsAccount saver1(2000.00);
    SavingsAccount saver2(200.00);

    // Set interest rate to 4%
    SavingsAccount::modifyInterestRate(0.04);

    saver1.calculateMonthlyInterest();
    saver2.calculateMonthlyInterest();

    cout << "Balances after applying 4% annual interest rate:" << endl;
    cout << "Saver 1 - ";
    saver1.displayBalance();
    cout << "Saver 2 - ";
    saver2.displayBalance();

    // Change interest rate to 5%
    SavingsAccount::modifyInterestRate(0.05);

    saver1.calculateMonthlyInterest();
    saver2.calculateMonthlyInterest();

    cout << "\nBalances after applying 5% annual interest rate:" << endl;
    cout << "Saver 1 - ";
    saver1.displayBalance();
    cout << "Saver 2 - ";
    saver2.displayBalance();

    return 0;
}

```

OUTPUT:

```
y\Desktop\abhinav.exe x x + v
Balances after applying 4% annual interest rate:
Saver 1 - Balance: Rs 2006.67
Saver 2 - Balance: Rs 3010

Balances after applying 5% annual interest rate:
Saver 1 - Balance: Rs 2015.03
Saver 2 - Balance: Rs 3022.54

-----
Process exited after 2.173 seconds with return value 0
Press any key to continue . . .
```

14. Create a class Complex having two int type variable named real & img denoting real and imaginary part respectively of a complex number. Overload +, -, == operator to add, to subtract and to compare two complex numbers being denoted by the two complex type objects.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include<iostream>
using namespace std;
class Complex {
public:
    int real, img;
    Complex(int r=0,int i=0){ real=r; img=i; }
    Complex operator+(Complex c){
        return Complex(real+c.real, img+c.img);
    }

    Complex operator-(Complex c){
        return Complex(real-c.real, img-c.img);
    }

    bool operator==(Complex c){
        return (real==c.real && img==c.img);
    }
    void show(){
        cout<<real<<" + "<<img<<"i";
    }
};

int main(){
    Complex a(3,4), b(1,2);
    Complex s=a+b;
    Complex d=a-b;
    cout<<"Test Case 1\n";
    a.show(); cout<<" + "; b.show(); cout<<" = "; s.show(); cout<<"\n";
    a.show(); cout<<" - "; b.show(); cout<<" = "; d.show(); cout<<"\n";
    cout<<"Equal? "<<(a==b?"Yes":"No")<<"\n\n";
    Complex x(5,7), y(5,7);
    Complex p=x+y;
    Complex q=x-y;
    cout<<"Test Case 2\n";
    x.show(); cout<<" + "; y.show(); cout<<" = "; p.show(); cout<<"\n";
    x.show(); cout<<" - "; y.show(); cout<<" = "; q.show(); cout<<"\n";
    cout<<"Equal? "<<(x==y?"Yes":"No")<<"\n";
}
```

```
C:\N y\Desktop\abhinav.exe x x + v
Test Case 1
3 + 4i + 1 + 2i = 4 + 6i
3 + 4i - 1 + 2i = 2 + 2i
Equal? No

Test Case 2
5 + 7i + 5 + 7i = 10 + 14i
5 + 7i - 5 + 7i = 0 + 0i
Equal? Yes

-----
Process exited after 2.773 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-8

15. Using the concept of operator overloading. Implement a program to overload the following:

a. Unary – b. Unary ++ preincrement, postincrement c. Unary -- predecrement, postdecrement.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include<iostream>
using namespace std;

class Num {
public:
    int x;
    Num(int v=0){ x=v; }

    Num operator-(){
        return Num(-x);
    }

    Num operator++(){
        ++x;
        return *this;
    }

    Num operator++(int){
        Num t=*this;
        x++;
        return t;
    }

    Num operator--(){
        --x;
        return *this;
    }

    Num operator--(int){
        Num t=*this;
        x--;
        return t;
    }
};

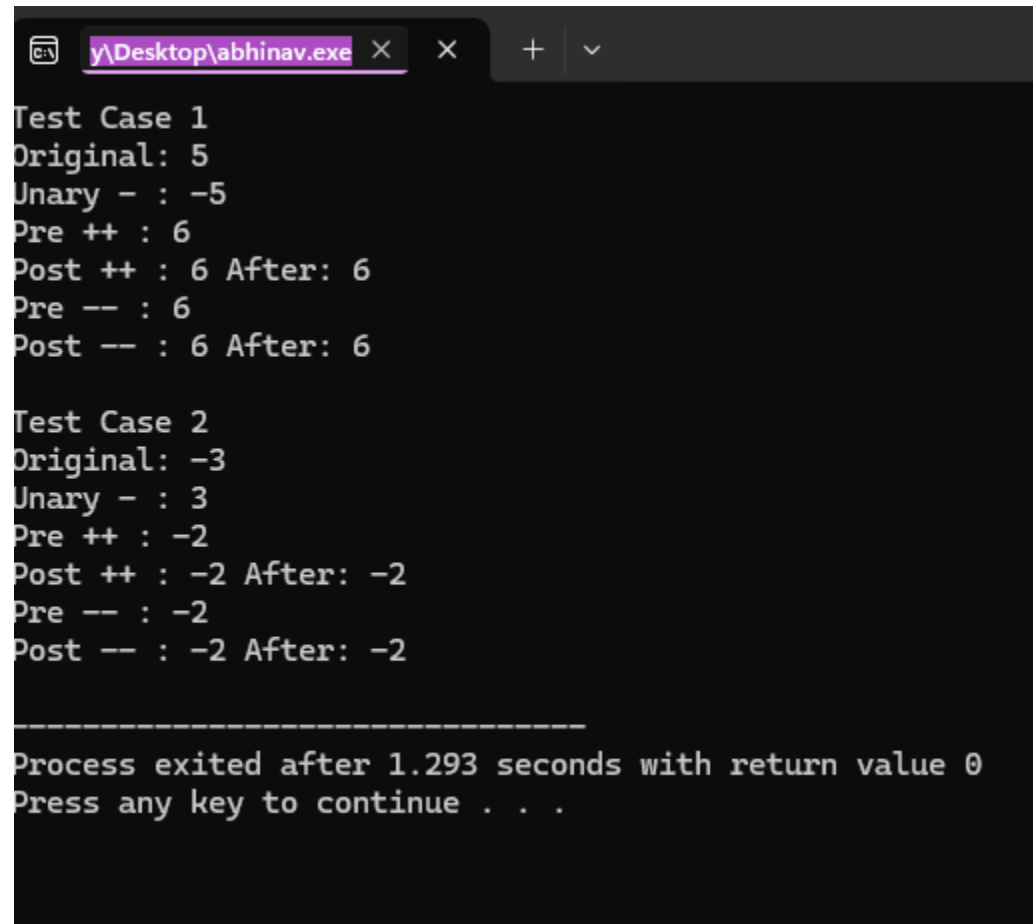
int main(){
```

```
Num a(5);
cout<<"Test Case 1\n";
cout<<"Original: "<<a.x<<"\n";
cout<<"Unary - : "<<(-a).x<<"\n";
cout<<"Pre ++ : "<<(++a).x<<"\n";
cout<<"Post ++ : "<<(a++).x<<" After: "<<a.x<<"\n";
cout<<"Pre -- : "<<(--a).x<<"\n";
cout<<"Post -- : "<<(a--).x<<" After: "<<a.x<<"\n\n";
```

```
Num b(-3);
cout<<"Test Case 2\n";
cout<<"Original: "<<b.x<<"\n";
cout<<"Unary - : "<<(-b).x<<"\n";
cout<<"Pre ++ : "<<(++b).x<<"\n";
cout<<"Post ++ : "<<(b++).x<<" After: "<<b.x<<"\n";
cout<<"Pre -- : "<<(--b).x<<"\n";
cout<<"Post -- : "<<(b--).x<<" After: "<<b.x<<"\n";
```

```
}
```

OUTPUT:



```
y\Desktop\abhinav.exe x x + v
Test Case 1
Original: 5
Unary - : -5
Pre ++ : 6
Post ++ : 6 After: 6
Pre -- : 6
Post -- : 6 After: 6

Test Case 2
Original: -3
Unary - : 3
Pre ++ : -2
Post ++ : -2 After: -2
Pre -- : -2
Post -- : -2 After: -2

-----
Process exited after 1.293 seconds with return value 0
Press any key to continue . . .
```

16. Using the concept of operator overloading. Implement a program to overload the following:
With the help of friend function a. Unary – b. Unary ++ preincrement, postincrement c. Unary --
predecrement, postdecrement.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include<iostream>
```

```
using namespace std;
```

```
class Num {
public:
    int x;
    Num(int v=0){ x=v; }

    friend Num operator-(Num n){
        return Num(-n.x);
    }

    friend Num operator++(Num &n){
        ++n.x;
        return n;
    }

    friend Num operator++(Num &n, int){
        Num t=n;
        n.x++;
        return t;
    }

    friend Num operator--(Num &n){
        --n.x;
        return n;
    }

    friend Num operator--(Num &n, int){
        Num t=n;
        n.x--;
        return t;
    }
};

int main(){
    Num a(10);
    cout<<"Test Case 1\n";
    cout<<"Original: "<<a.x<<"\n";
```

```
cout<<"Unary - : "<<(-a).x<<"\n";
cout<<"Pre ++ : "<<(++a).x<<"\n";
cout<<"Post ++ : "<<(a++).x<<" After: "<<a.x<<"\n";
cout<<"Pre -- : "<<(--a).x<<"\n";
cout<<"Post -- : "<<(a--).x<<" After: "<<a.x<<"\n\n";
```

```
Num b(-4);
cout<<"Test Case 2\n";
cout<<"Original: "<<b.x<<"\n";
cout<<"Unary - : "<<(-b).x<<"\n";
cout<<"Pre ++ : "<<(++b).x<<"\n";
cout<<"Post ++ : "<<(b++).x<<" After: "<<b.x<<"\n";
cout<<"Pre -- : "<<(--b).x<<"\n";
cout<<"Post -- : "<<(b--).x<<" After: "<<b.x<<"\n";
```

```
}
```

OUTPUT:

```
y\Deskto\abhinav.exe x x + v
Test Case 1
Original: 10
Unary - : -10
Pre ++ : 11
Post ++ : 11 After: 11
Pre -- : 11
Post -- : 11 After: 11

Test Case 2
Original: -4
Unary - : 4
Pre ++ : -3
Post ++ : -3 After: -3
Pre -- : -3
Post -- : -3 After: -3

-----
Process exited after 4.265 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-9

17. Create a Base class that consists of private, protected and public data members and member functions. Try using different access modifiers for inheriting Base class to the Derived class and create a table that summarizes the above three modes (when derived in public, protected and private modes) and shows the access specifier of the members of base class in the Derived class.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include<iostream>
using namespace std;
```

```
class Base {
private:
    int a;
protected:
    int b;
public:
    int c;
    Base(){ a=1; b=2; c=3; }
};
```

```
class DerivedPublic : public Base {
public:
    void show(){
        // a not accessible
        cout<<b<<" "<<c<<"\n";
    }
};
```

```
class DerivedProtected : protected Base {
public:
    void show(){
        // a not accessible
        cout<<b<<" "<<c<<"\n";
    }
};
```

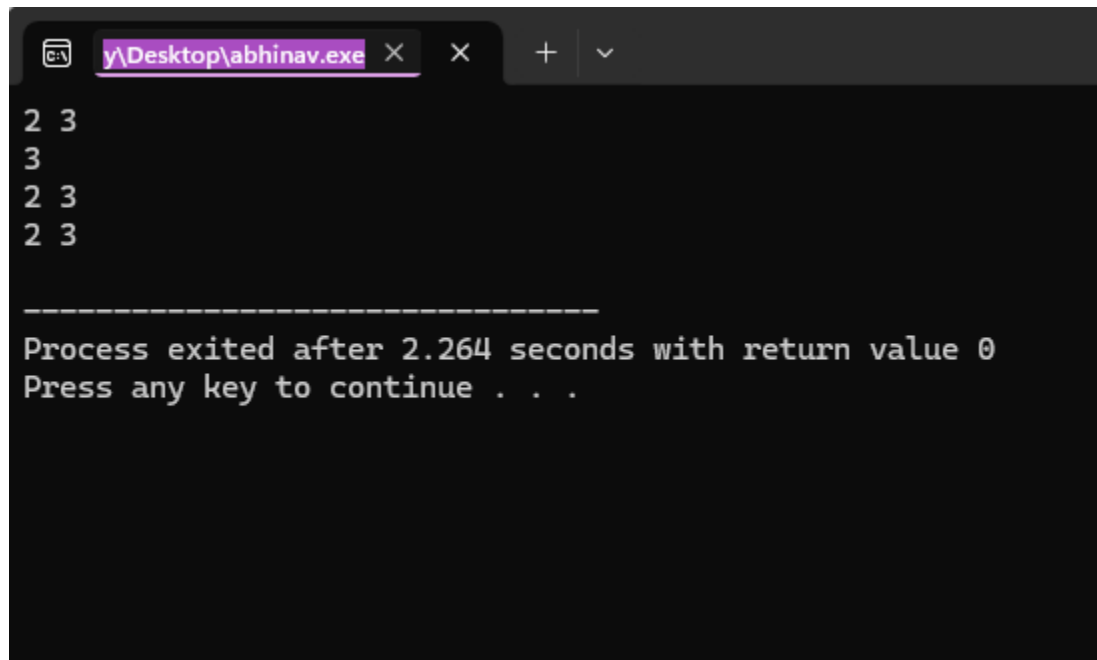
```
class DerivedPrivate : private Base {
public:
    void show(){
        // a not accessible
        cout<<b<<" "<<c<<"\n";
    }
};
```

```
int main(){
    DerivedPublic p;
    p.show();
    cout<<p.c<<"\n";

    DerivedProtected q;
    q.show();
    // q.c not accessible

    DerivedPrivate r;
    r.show();
    // r.c not accessible
}
```

OUTPUT:



```
y:\Desktop\abhinav.exe x x + v
2 3
3
2 3
2 3

-----
Process exited after 2.264 seconds with return value 0
Press any key to continue . . .
```

18. You are given three classes A, B and C. All three classes implement their own version of func. In class A, func multiplies the value passed as a parameter by 2. In class B, func multiplies the value passed as a parameter by 3. In class C, func multiplies the value passed as a parameter by 5. You are given class D such that You need to modify the class D and implement the function update_val which sets D's val to new_val by manipulating the value by only calling the func defined in classes A, B and C. It is guaranteed that new_val has only 2, 3 and 5 as its prime factors. Implement class D's function update_val. This function should update D's val only by calling A, B and C's func.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include<iostream>
```

```
using namespace std;
```

```
class A {  
public:  
    int func(int v){ return v*2; }  
};
```

```
class B {  
public:  
    int func(int v){ return v*3; }  
};
```

```
class C {  
public:  
    int func(int v){ return v*5; }  
};
```

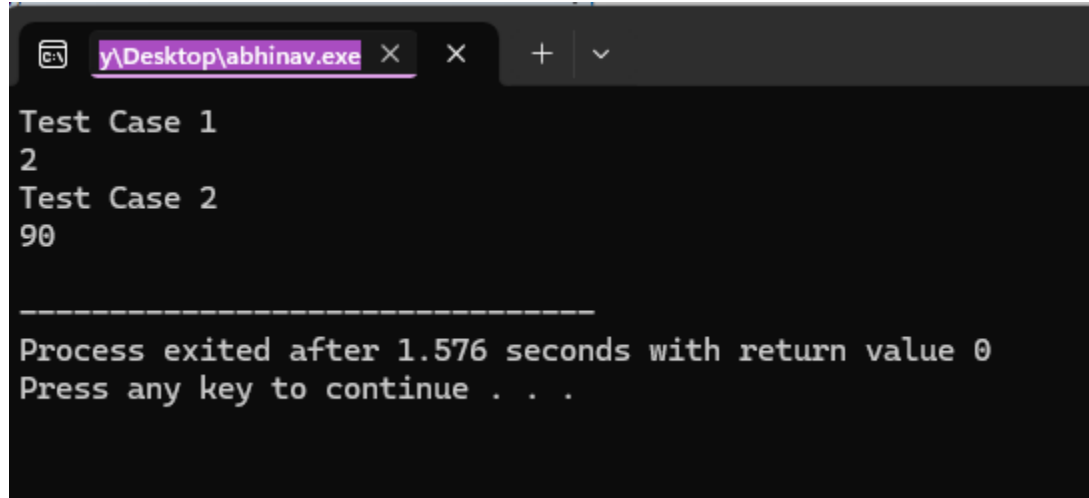
```
class D {  
public:  
    int val;  
    D(){ val = 1; }  
    A a;  
    B b;  
    C c;
```

```
    void update_val(int new_val){  
        while(new_val % 2 == 0){  
            val = a.func(val);  
            new_val /= 2;  
        }  
        while(new_val % 3 == 0){  
            val = b.func(val);  
            new_val /= 3;
```

```
    }  
    while(new_val % 5 == 0){  
        val = c.func(val);  
        new_val /= 5;  
    }  
}  
};
```

```
int main(){  
    D d;  
  
    cout<<"Test Case 1\n";  
    d.update_val(2);  
    cout<<d.val<<"\n";  
  
    D d2;  
    cout<<"Test Case 2\n";  
    d2.update_val(90);  
    cout<<d2.val<<"\n";  
}
```

OUTPUT:



```
C:\> y\Desktop\abhinav.exe
Test Case 1
2
Test Case 2
90

-----
Process exited after 1.576 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-10

19. Create a class called Student that contains the data members like age, name, enroll_no, marks. Create another class called Faculty that contains data members like facultyName, facultyCode, salary,deptt, age, experience, gender. Create the function display() in both the classes to display the respective information. The derived Class Person demonstrates multiple inheritance. The program should be able to call both the base classes and displays their information. Remove the ambiguity (When we have exactly same variables or same methods in both the base classes, which one will be called?) by proper mechanism.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include<iostream>
```

```
using namespace std;
```

```
class Student {
public:
    int age, enroll_no, marks;
    string name;

    Student(int a, string n, int e, int m) {
        age=a; name=n; enroll_no=e; marks=m;
    }

    void display() {
        cout<<"Student Info\n";
        cout<<"Name: "<<name<<"\nAge: "<<age<<"\nEnroll No: "<<enroll_no<<"\nMarks: "<<marks<<"\n\n";
    }
};
```

```
class Faculty {
public:
    string facultyName, deptt, gender;
    int facultyCode, age, experience, salary;

    Faculty(string fn, int fc, string d, int s, int a, int exp, string g) {
        facultyName=fn; facultyCode=fc; deptt=d; salary=s; age=a; experience=exp; gender=g;
    }

    void display() {
        cout<<"Faculty Info\n";
        cout<<"Name: "<<facultyName<<"\nAge: "<<age<<"\nCode: "<<facultyCode;
        cout<<"\nDept: "<<deptt<<"\nSalary: "<<salary<<"\nExperience: "<<experience<<"
years\nGender: "<<gender<<"\n\n";
    }
};
```

```
    }  
};
```

```
class Person : public Student, public Faculty {
```

```
public:
```

```
    Person() : Student(0,"",0,0), Faculty("",0,"",0,0,0,"") {}
```

```
    Person(int sa, string sn, int se, int sm,  
           string fn, int fc, string fd, int fs, int fa, int fe, string fg)  
    : Student(sa,sn,se,sm), Faculty(fn,fc,fd,fs,fa,fe,fg) {}
```

```
    void showAll() {  
        Student::display();  
        Faculty::display();  
    }
```

```
};
```

```
int main() {
```

```
    cout<<"Test Case 1\n";
```

```
    Person p1(20,"Kritika",101,89,"Dr. Mehta",501,"CSE",60000,45,20,"Male");
```

```
    p1.showAll();
```

```
    cout<<"Test Case 2\n";
```

```
    Person p2(19,"Arjun",102,92,"Prof. Kapoor",502,"ECE",65000,50,22,"Female");
```

```
    p2.showAll();
```

```
}
```



y:\Desktop\abhinav.exe



Test Case 1

Student Info

Name: Kritika

Age: 20

Enroll No: 101

Marks: 89

Faculty Info

Name: Dr. Mehta

Age: 45

Code: 501

Dept: CSE

Salary: 60000

Experience: 20 years

Gender: Male

Test Case 2

Student Info

Name: Arjun

Age: 19

Enroll No: 102

Marks: 92

Faculty Info

Name: Prof. Kapoor

Age: 50

Code: 502

Dept: ECE

Salary: 65000

Experience: 22 years

20. Implement a real case scenario by a proper C++ code to provide the solution to Diamond Problem in C++.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include<iostream>
using namespace std;

class Person {
public:
    string name;
    int age;
    Person(string n="", int a=0) { name=n; age=a; }
    void showPerson() { cout<<"Name: "<<name<<"\nAge: "<<age<<"\n"; }
};

class Student : virtual public Person {
public:
    string course;
    Student(string n="", int a=0, string c="") : Person(n,a) { course=c; }
    void showStudent() { cout<<"Course: "<<course<<"\n"; }
};

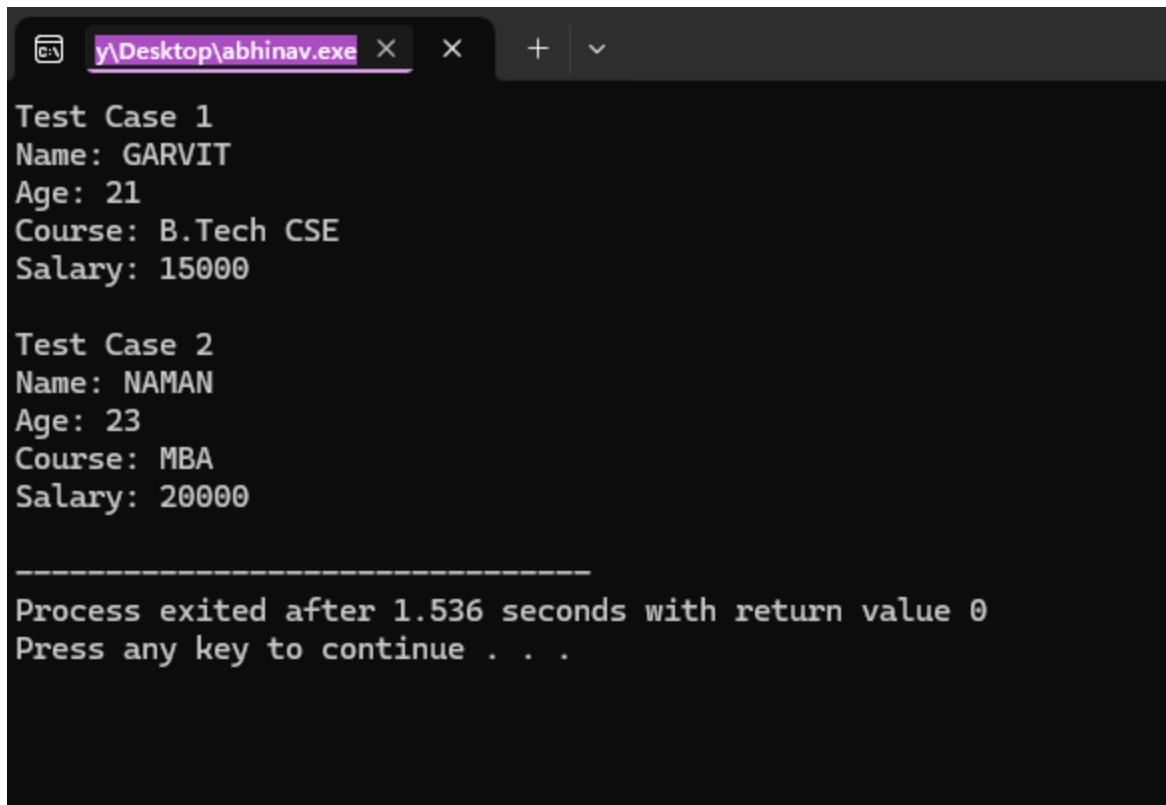
class Employee : virtual public Person {
public:
    int salary;
    Employee(string n="", int a=0, int s=0) : Person(n,a) { salary=s; }
    void showEmployee() { cout<<"Salary: "<<salary<<"\n"; }
};

class WorkingStudent : public Student, public Employee {
public:
    WorkingStudent(string n, int a, string c, int s)
    : Person(n,a), Student(n,a,c), Employee(n,a,s) {}

    void showAll() {
        showPerson();
        showStudent();
        showEmployee();
    }
};

int main() {
```

```
cout<<"Test Case 1\n";  
WorkingStudent w1("Kritika", 21, "B.Tech CSE", 15000);  
w1.showAll();  
cout<<"\n";  
  
cout<<"Test Case 2\n";  
WorkingStudent w2("Rahul", 23, "MBA", 20000);  
w2.showAll();  
}
```



```
Test Case 1
Name: GARVIT
Age: 21
Course: B.Tech CSE
Salary: 15000

Test Case 2
Name: NAMAN
Age: 23
Course: MBA
Salary: 20000

-----
Process exited after 1.536 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-11

21. Create a base class called shape. Use this class to store two double type values that could be used to compute the area of figures. Derive two specific classes called triangle and rectangle from base shape. Add to the base class, a member function get_data() to initialize base class data members and another member function display_area() to compute and display the area of figures. Make display_area() as a virtual function and redefine this function in the derived class to suit their requirements. Using these three classes, design a program that will accept dimensions of a triangle or a rectangle interactively and display the area. Remember the two values given as input will be treated as lengths of two sides in the case of rectangles and as base and height in the case of triangle and used as follows: Area of rectangle = $x * y$ Area of triangle = $\frac{1}{2} * x * y$.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include<iostream>
using namespace std;

class Shape {
public:
    double x, y;
    void get_data(double a, double b){
        x=a; y=b;
    }
    virtual void display_area(){
        cout<<"Area\n";
    }
};

class Triangle : public Shape {
public:
    void display_area(){
        cout<<"Area of Triangle: "<<0.5*x*y<<"\n";
    }
};

class Rectangle : public Shape {
public:
    void display_area(){
        cout<<"Area of Rectangle: "<<x*y<<"\n";
    }
};

int main(){
    Triangle t;
```

```
Rectangle r;
```

```
cout<<"Test Case 1\n";
```

```
t.get_data(10,5);
```

```
t.display_area();
```

```
cout<<"Test Case 2\n";
```

```
r.get_data(4,6);
```

```
r.display_area();
```

```
double a,b;
```

```
int ch;
```

```
cout<<"\nEnter 1 for Triangle, 2 for Rectangle: ";
```

```
cin>>ch;
```

```
cout<<"Enter two values: ";
```

```
cin>>a>>b;
```

```
Shape *s;
```

```
if(ch==1){
```

```
    s=&t;
```

```
    s->get_data(a,b);
```

```
    s->display_area();
```

```
} else {
```

```
    s=&r;
```

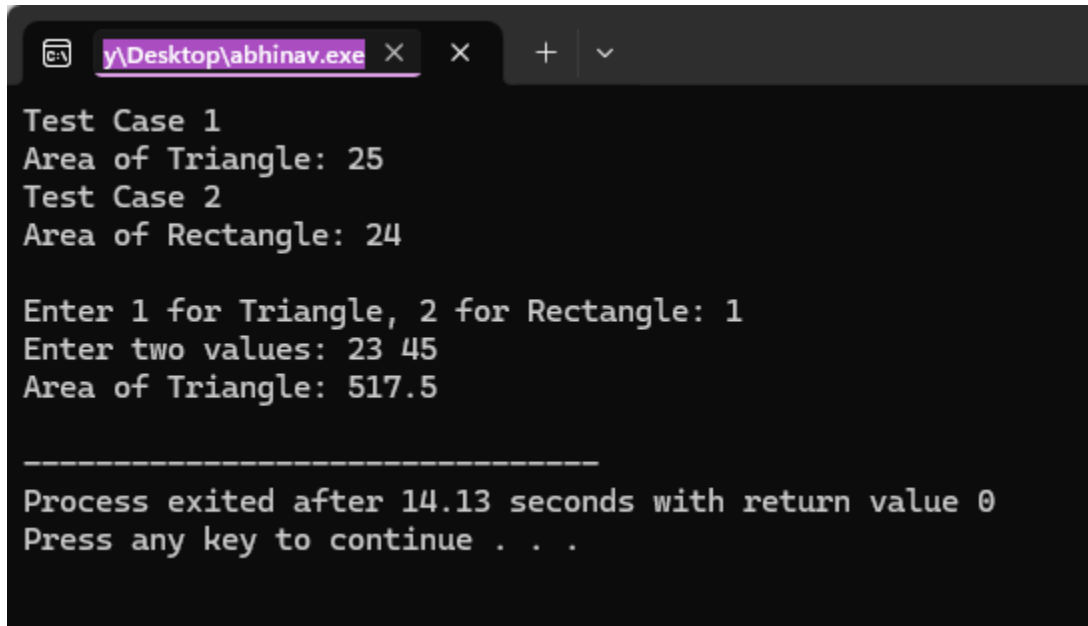
```
    s->get_data(a,b);
```

```
    s->display_area();
```

```
}
```

```
}
```

OUTPUT:



```
y\Desktop\abhinav.exe x x + v
Test Case 1
Area of Triangle: 25
Test Case 2
Area of Rectangle: 24

Enter 1 for Triangle, 2 for Rectangle: 1
Enter two values: 23 45
Area of Triangle: 517.5

-----
Process exited after 14.13 seconds with return value 0
Press any key to continue . . .
```

22. Create a base class called CAL_AREA(Abstract). Use this class to store float type values that could be used to compute the volume of figures. Derive two specific classes called cone, hemisphere and cylinder from the base CAL_AREA. Add to the base class, a member function getdata () to initialize base class data members and another member function display volume () to compute and display the volume of figures. Make display volume () as a pure virtual function and redefine this function in the derived classes to suit their requirements. Using these three classes, design a program that will accept dimensions of a cone, cylinder and hemisphere interactively and display the volumes. Remember values given as input will be and used as follows: 2023-24 and 2024-25 onwards Volume of cone = $(1/3)\pi r^2 h$ Volume of hemisphere = $(2/3)\pi r^3$ Volume of cylinder = $\pi r^2 h$.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE:

```
#include <iostream>
#define pi 3.14159
using namespace std;

class CAL_AREA {
public:
    float r,h;
    void getdata(float x, float y=0){
        r = x;
        h = y;
    }
    virtual void display_volume() = 0;
};

class cone : public CAL_AREA {
public:
    void display_volume(){
        float v = (1.0/3) * pi * r * r * h;
        cout<<"Volume of Cone = "<<v<<endl;
    }
};

class cylinder : public CAL_AREA {
public:
    void display_volume(){
        float v = pi * r * r * h;
        cout<<"Volume of Cylinder = "<<v<<endl;
    }
};

class hemisphere : public CAL_AREA {
```

```

public:
    void display_volume(){
        float v = (2.0/3) * pi * r * r * r;
        cout<<"Volume of Hemisphere = "<<v<<endl;
    }
};

int main(){
    cone c;
    cylinder cy;
    hemisphere h;

    float r, ht;

    cout<<"Enter radius and height of Cone: ";
    cin>>r>>ht;
    c.getdata(r, ht);
    c.display_volume();

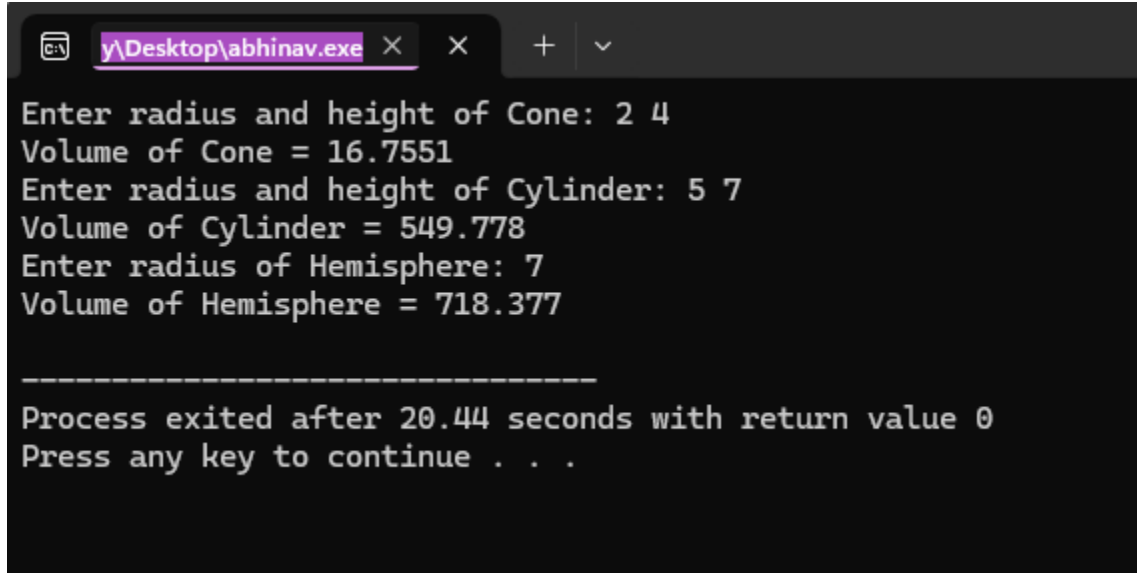
    cout<<"Enter radius and height of Cylinder: ";
    cin>>r>>ht;
    cy.getdata(r, ht);
    cy.display_volume();

    cout<<"Enter radius of Hemisphere: ";
    cin>>r;
    h.getdata(r);
    h.display_volume();

    return 0;
}

```

OUTPUT:



```
y:\Desktop\abhinav.exe x x + v
Enter radius and height of Cone: 2 4
Volume of Cone = 16.7551
Enter radius and height of Cylinder: 5 7
Volume of Cylinder = 549.778
Enter radius of Hemisphere: 7
Volume of Hemisphere = 718.377

-----
Process exited after 20.44 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-12

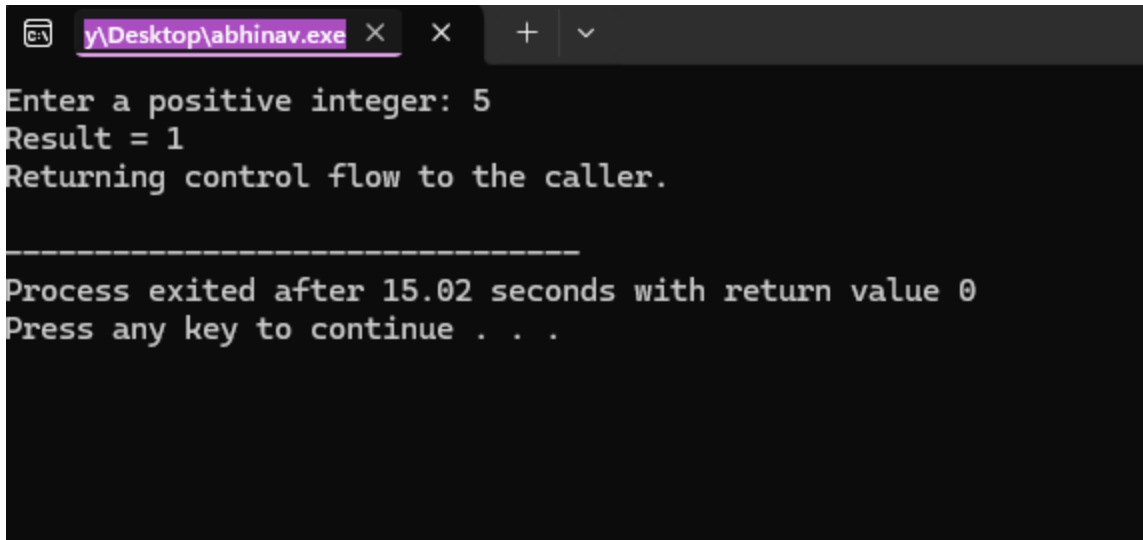
23. The task is to debug the existing code to successfully execute all provided test files. You are required to extend the existing code so that it handles the `std::invalid_argument` exception properly. More specifically, you have to extend the implementation of the `process_input` function. It takes integer `n` as an argument and has to work as follows: 1.It calls function `largest_proper_divisor(n)`. 2.If this call returns a value without raising an exception, it should print in a single line `result=d` where `d` is the returned value. 3.Otherwise, if the call raises an `invalid_argument` exception, it has to print in a single line the string representation of the raised exception, i.e., its message. 4.Finally, no matter if the exception is raised or not, it should print in a single line returning control flow to the caller after any other previously printed output.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

```
#include <iostream>
#include <stdexcept>
using namespace std;
int largest_proper_divisor(int n) {
    if (n == 0) {
        throw invalid_argument("Argument should be greater than 1");
    }
    if (n == 1) {
        throw invalid_argument("Argument should be greater than 1");
    }
    int largest = 1;
    for (int i = 1; i <= n / 2; ++i) {
        if (n % i == 0) {
            largest = i;
        }
    }
    return largest;
}
void process_input(int n) {
    try {
        int d = largest_proper_divisor(n);
        cout << "result=" << d << endl;
    }
    catch (const invalid_argument& e) {
        cout << e.what() << endl;
    }
    cout << "returning control flow to the caller" << endl;
}
int main() {
    int n;
    cin >> n;
    process_input(n);
    return 0;
}
```



```
y\Desktop\abhinav.exe X X + v
Enter a positive integer: 5
Result = 1
Returning control flow to the caller.

-----
Process exited after 15.02 seconds with return value 0
Press any key to continue . . .
```

24. Write a C++ program to retrieve a particular record from a file .
Name-ABHINAV SINGH
Section-E2
Roll no.-2

SOURCE CODE :

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of students to store: ";
    cin >> n;

    // Create and open file for writing
    ofstream outfile("student.txt");

    if (!outfile) {
        cout << "Error: Cannot create file!" << endl;
        return 0;
    }

    int roll;
    string name;
    float marks;

    // Write student data into file
    cout << "\nEnter student details (Roll Name Marks):" << endl;
    for (int i = 0; i < n; i++) {
        cout << "Student " << i + 1 << ": ";
        cin >> roll >> name >> marks;
        outfile << roll << " " << name << " " << marks << endl;
    }

    outfile.close();
    cout << "\nData successfully written to file 'student.txt'!" << endl;

    // Now search for a particular student
    int searchRoll;
    cout << "\nEnter roll number to search: ";
    cin >> searchRoll;

    ifstream infile("student.txt");
```

```
if(!infile) {
    cout << "Error: Cannot open file for reading!" << endl;
    return 0;
}

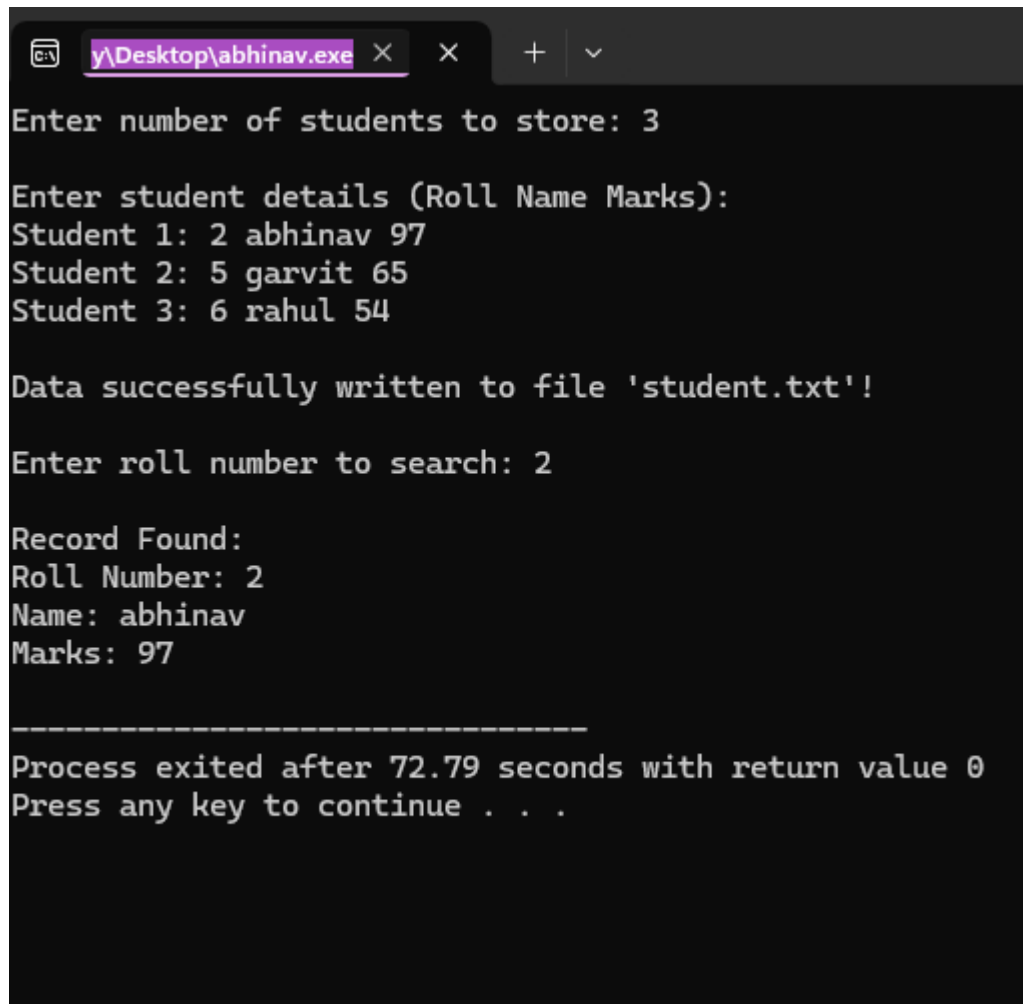
bool recordFound = false;

// Read each record and compare roll numbers
while (infile >> roll >> name >> marks) {
    if(roll == searchRoll) {
        cout << "\nRecord Found:" << endl;
        cout << "Roll Number: " << roll << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
        recordFound = true;
        break;
    }
}

if(!recordFound) {
    cout << "\nRecord not found!" << endl;
}

infile.close();
return 0;
}
```

OUTPUT:



```
y:\Desktop\abhinav.exe X X + v
Enter number of students to store: 3

Enter student details (Roll Name Marks):
Student 1: 2 abhinav 97
Student 2: 5 garvit 65
Student 3: 6 rahul 54

Data successfully written to file 'student.txt'!

Enter roll number to search: 2

Record Found:
Roll Number: 2
Name: abhinav
Marks: 97

-----
Process exited after 72.79 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-13

25. Write a C++ program to write the data such as student_name, student_id and student_branch and show it to the console .

Name-ABHINAV SINGH

Section-E2

Roll no.-2

SOURCE CODE :

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

```
int main() {
```

```
    string name, branch;
```

```
    int id;
```

```
    // Taking Student Details
```

```
    cout << "Enter Student Name: ";
```

```
    getline(cin, name);
```

```
    cout << "Enter Student ID: ";
```

```
    cin >> id;
```

```
    cout << "Enter Student Branch: ";
```

```
    cin >> branch;
```

```
    // Writing to file
```

```
    ofstream outFile("student.txt");
```

```
    if (!outFile) {
```

```
        cout << "Error opening file!" << endl;
```

```
        return 0;
```

```
    }
```

```
    outFile << name << endl;
```

```
    outFile << id << endl;
```

```
    outFile << branch << endl;
```

```
    outFile.close();
```

```
    // Reading from file
```

```
    ifstream inFile("student.txt");
```

```
    if (!inFile) {
```

```
        cout << "Error reading file!" << endl;
```

```
        return 0;
```

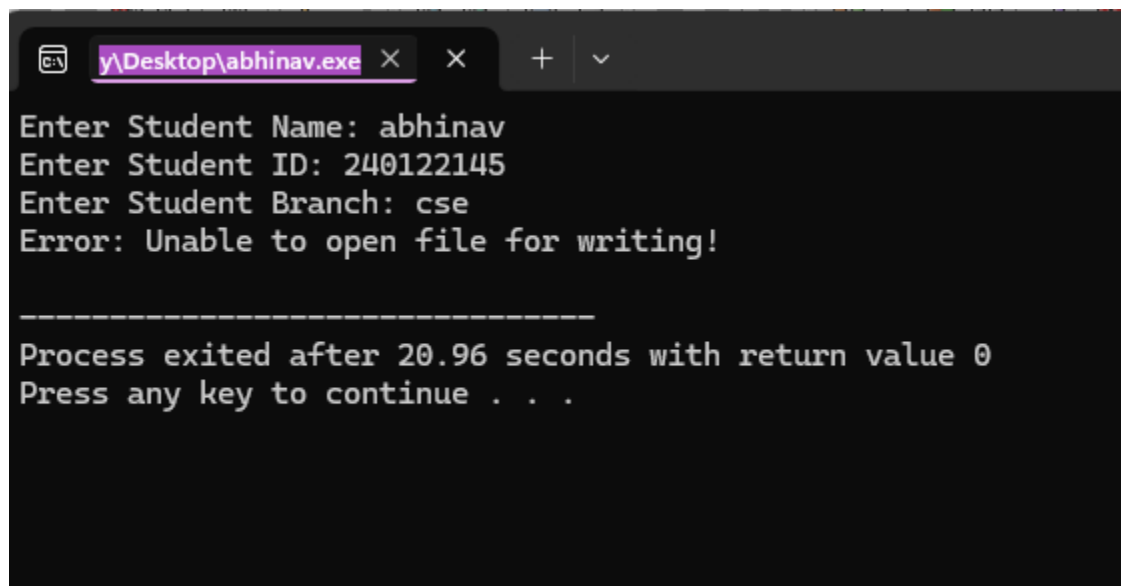
```
    }
```

```
cout << "\n--- Student Details from File ---\n";
string fileName, fileBranch;
int fileId;

getline(inFile, fileName);
inFile >> fileId;
inFile >> fileBranch;

cout << "Student Name: " << fileName << endl;
cout << "Student ID: " << fileId << endl;
cout << "Student Branch: " << fileBranch << endl;

inFile.close();
return 0;
}
```



```
Enter Student Name: abhinav
Enter Student ID: 240122145
Enter Student Branch: cse
Error: Unable to open file for writing!

-----
Process exited after 20.96 seconds with return value 0
Press any key to continue . . .
```

26. Write a program in C++ to use open() and eof() function.

Name-ABHINAV SINGH

Section-E2

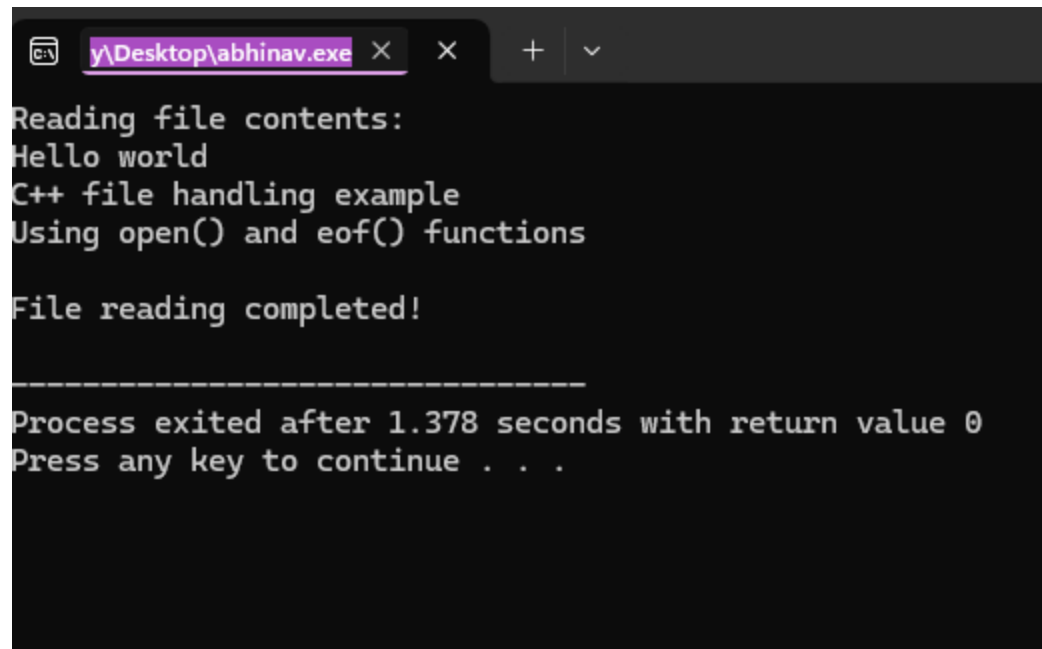
Roll no.-2

```
#include <iostream>
```

```
#include <fstream> // For file operations
```

```
using namespace std;
```

```
int main() {  
    // Step 1: Create and write data into a file  
    ofstream outFile;  
    outFile.open("example.txt"); // Create or overwrite file  
  
    if (!outFile) {  
        cout << "Error creating file!" << endl;  
        return 1;  
    }  
  
    outFile << "Hello world\n";  
    outFile << "C++ file handling example\n";  
    outFile << "Using open() and eof() functions\n";  
    outFile.close(); // Close the file after writing  
  
    // Step 2: Open the same file for reading  
    ifstream inFile;  
    inFile.open("example.txt");  
  
    if (!inFile) {  
        cout << "Error opening file for reading!" << endl;  
        return 1;  
    }  
  
    cout << "Reading file contents:\n";  
  
    string line;  
    while (!inFile.eof()) { // Loop until end of file  
        getline(inFile, line);  
        if (inFile.eof() && line.empty()) break; // avoid printing extra empty line  
        cout << line << endl;  
    }  
  
    inFile.close(); // Close the file  
    cout << "\nFile reading completed!" << endl;  
  
    return 0;  
}
```



```
Reading file contents:
Hello world
C++ file handling example
Using open() and eof() functions

File reading completed!

-----
Process exited after 1.378 seconds with return value 0
Press any key to continue . . .
```

PRACTICAL-14

27. Templates are the foundation of generic programming, which involves writing code in a way that is independent of any particular type. Write a program that can create a list (create a class list) of given type (int, float, char etc.) and perform insertion and deletion on list object.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

```
#include <iostream>
```

```
using namespace std;
```

```
template <class T>
```

```
class List {
```

```
private:
```

```
    T arr[100];
```

```
    int size;
```

```
public:
```

```
    List() {
```

```
        size = 0;
```

```
    }
```

```
    void insert(T element) {
```

```
        if (size < 100) {
```

```
            arr[size] = element;
```

```
            size++;
```

```
            cout << "Inserted: " << element << endl;
```

```
        } else {
```

```
            cout << "List is full!" << endl;
```

```
        }
```

```
    }
```

```
    void remove() {
```

```
        if (size > 0) {
```

```
            cout << "Deleted: " << arr[size - 1] << endl;
```

```
            size--;
```

```
        } else {
```

```
            cout << "List is empty!" << endl;
```

```
        }
```

```
    }
```

```
    void display() {
```

```
        if (size == 0) {
```

```
            cout << "List is empty!" << endl;
```

```
            return;
```

```
        }
```

```

        cout << "List elements: ";
        for (int i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

int main() {
    cout << "--- Integer List ---" << endl;
    List<int> intList;
    intList.insert(10);
    intList.insert(20);
    intList.insert(2);
    intList.display();
    intList.remove();
    intList.display();

    cout << "\n--- Float List ---" << endl;
    List<float> floatList;
    floatList.insert(1.1);
    floatList.insert(2.2);
    floatList.display();

    cout << "\n--- Character List ---" << endl;
    List<char> charList;
    charList.insert('A');
    charList.insert('B');
    charList.display();
    charList.remove();
    charList.display();

    return 0;
}

```



y\Desktop\abhinav.exe



--- Integer List ---

Inserted: 10

Inserted: 20

Inserted: 2

List elements: 10 20 2

Deleted: 2

List elements: 10 20

--- Float List ---

Inserted: 1.1

Inserted: 2.2

List elements: 1.1 2.2

--- Character List ---

Inserted: A

Inserted: B

List elements: A B

Deleted: B

List elements: A

Process exited after 15.67 seconds with return value 0

Press any key to continue . . .

28. Construct a C++ program to demonstrate different methods of List, Vector and Map in STL (Standard Template Library).

Name-ABHINAV SINGH

Section-E2

Roll no.-2

```
#include <iostream>
```

```
#include <list>
```

```
#include <vector>
```

```
#include <map>
```

```
using namespace std;
```

```
int main() {  
    cout << "--- LIST ---" << endl;  
    list<int> l;  
    l.push_back(10);  
    l.push_back(20);  
    l.push_front(5);  
    cout << "List elements: ";  
    for (list<int>::iterator it = l.begin(); it != l.end(); ++it)  
        cout << *it << " ";  
    cout << endl;  
    l.pop_front();  
    cout << "After pop_front: ";  
    for (list<int>::iterator it = l.begin(); it != l.end(); ++it)  
        cout << *it << " ";  
    cout << endl;  
  
    cout << "\n--- VECTOR ---" << endl;  
    vector<int> v;  
    v.push_back(1);  
    v.push_back(2);  
    v.push_back(3);  
    cout << "Vector elements: ";  
    for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)  
        cout << *it << " ";  
    cout << endl;  
    v.pop_back();  
    cout << "After pop_back: ";  
    for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)  
        cout << *it << " ";  
    cout << endl;  
    cout << "Size: " << v.size() << endl;  
    cout << "First element: " << v.front() << endl;  
    cout << "Last element: " << v.back() << endl;  
  
    cout << "\n--- MAP ---" << endl;
```

```
map<int, string> m;
m[1] = "Apple";
m[2] = "Banana";
m[3] = "Cherry";
cout << "Map elements:\n";
for (map<int, string>::iterator it = m.begin(); it != m.end(); ++it)
    cout << it->first << " -> " << it->second << endl;
m.erase(2);
cout << "After erase:\n";
for (map<int, string>::iterator it = m.begin(); it != m.end(); ++it)
    cout << it->first << " -> " << it->second << endl;
if (m.find(3) != m.end())
    cout << "Find key 3: Found" << endl;
else
    cout << "Find key 3: Not Found" << endl;

return 0;
}
```



y:\Desktop\abhinav.exe



--- LIST ---

List elements: 5 10 20

After pop_front: 10 20

--- VECTOR ---

Vector elements: 1 2 3

After pop_back: 1 2

Size: 2

First element: 1

Last element: 2

--- MAP ---

Map elements:

1 -> Apple

2 -> Banana

3 -> Cherry

After erase:

1 -> Apple

3 -> Cherry

Find key 3: Found

Process exited after 1.279 seconds with return value 0

Press any key to continue . . .

PRACTICAL-15

29. You are provided with a vector of N integers. Then, you are given 2 queries. For the first query, you are provided with 1 integer, which denotes a position in the vector. The value at this position in the vector needs to be erased. The next query consists of 2 integers denoting a range of the positions in the vector. The elements which fall under that range should be removed. The second query is performed on the updated vector which we get after performing the first query.

Name-ABHINAV SINGH

Section-E2

Roll no.-2

```
#include <iostream>
```

```
#include <vector>
```

```
#include <cstdio>
```

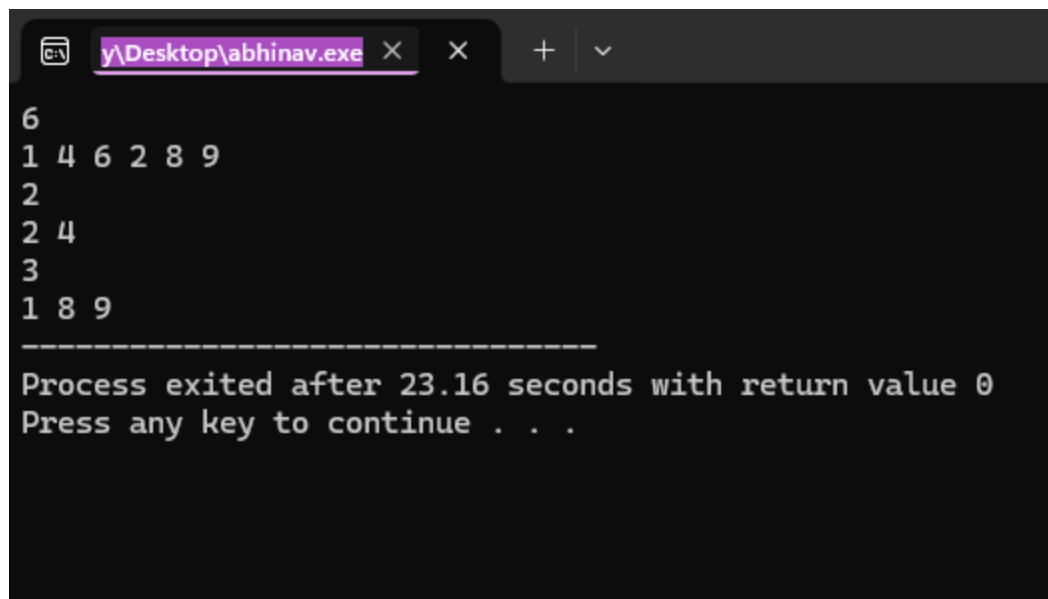
```
using namespace std;
```

```
int main() {
    int n;
    scanf("%d", &n);
    vector<int> v(n);
    for (int i = 0; i < n; i++)
        scanf("%d", &v[i]);

    int pos1;
    scanf("%d", &pos1);
    v.erase(v.begin() + (pos1 - 1));

    int start, end;
    scanf("%d %d", &start, &end);
    v.erase(v.begin() + (start - 1), v.begin() + (end - 1));

    printf("%d\n", (int)v.size());
    for (int i = 0; i < v.size(); i++)
        printf("%d ", v[i]);
    return 0;
}
```



```
6
1 4 6 2 8 9
2
2 4
3
1 8 9
-----
Process exited after 23.16 seconds with return value 0
Press any key to continue . . .
```